基于 GitHub 的开源软件协同开发

一、简介

（1）GitHub 中的 Pull Request（PR）

- Pull Request 是指用户在 GitHub 上向开源项目提出的代码合并请求，简称 PR。
- 它是一种协作方式，使得多个用户可以在同一代码库上共同开发并提交代码。

（2）Pull Request 的作用

- Pull Request 可以帮助项目贡献者管理代码贡献，避免出现冲突或错误的代码。
- 它可以促进代码审查和讨论，提高代码质量和可维护性。
- Pull Request 也是开源社区互相学习和分享的重要方式。

（3）操作流程

1. Fork 项目仓库到自己帐户下。
2. 创建新分支并进行修改。
3. 提交 Pull Request 并等待审核。
4. 审核通过后，代码将被合并到原项目中。

（4）注意事项

- 在提交 Pull Request 前，需要先与项目贡献者进行沟通和讨论。
- 遵循项目的代码规范和贡献指南。
- 及时回应审核者的反馈和建议。

（5）https or ssh

Bash

```bash
git clone https://github.com/weihuayi/fealpy.git # https
 git clone git@github.com:weihuayi/fealpy.git # ssh
```

- GitHub 中 https 和 ssh 是两种不同的协议，用于与远程仓库进行数据传输。
- 使用 https 协议时，需要输入用户名和密码进行身份验证。而使用 ssh 协议时，需要在本地生成一个公钥和私钥，并将公钥添加到 GitHub 账户中，以便进行身份验证。
- 使用 ssh 与远程仓库交互的优点是，无需每次都输入用户名和密码，同时也更加安全，因为私钥只保存在本地，不会被传输到远程服务器上。但是，生成和添加公钥的过程可能会比较繁琐。
- 使用 https 与远程仓库交互的优点是，操作比较简单，无需生成和添加公钥。但是，每次都需要输入用户名和密码进行身份验证，可能会比较麻烦。

> 💡 在国内推荐使用 `ssh` 协议访问 GitHub！

二、预备工作

（1）生成本地 SSH 密钥

1. 在本地生成自己的 SSH 密钥

输入第一个命令后一直按 Enter 键即可，不需要输入内容

邮箱那部分不用复制

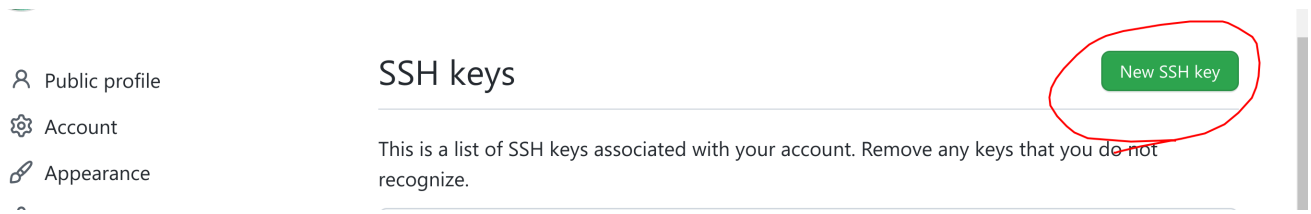
Bash

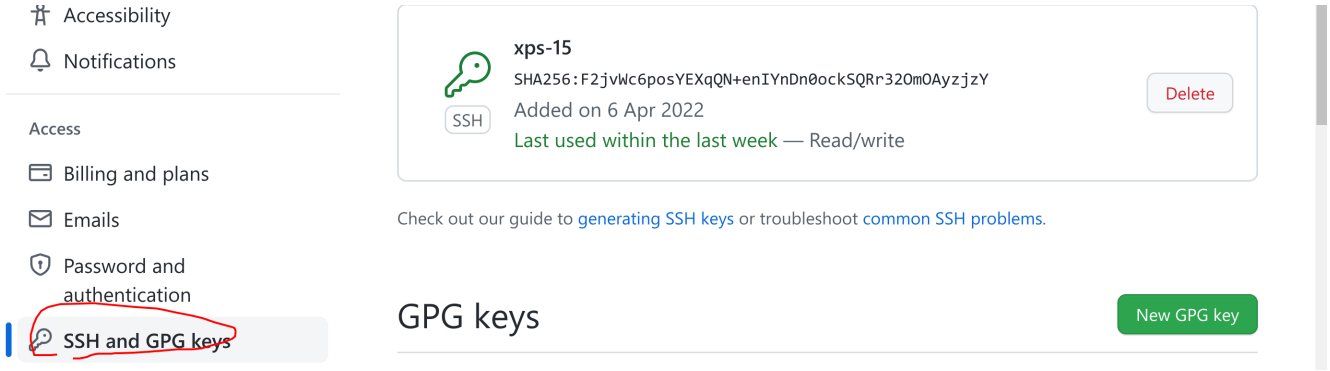
```bash
# <yours email> 换成自己注册 GitHub 帐户使用的邮箱
ssh-keygen -t ed25519 -C "<yours email>"
cat ~/.ssh/id_ed25519.pub
```

TODO：增加示例图片

---

2. 将密钥复制粘贴到个人帐号设置(不是仓库设置)的ssh中去





（2）Fork 原始仓库

第一步，我们需要将 https://github.com/weihuayi/fealpy 仓库 fork 到自己的 GitHub 帐号下。Fork 是 github 具有的功能（而不是 Git），具体操作如下：

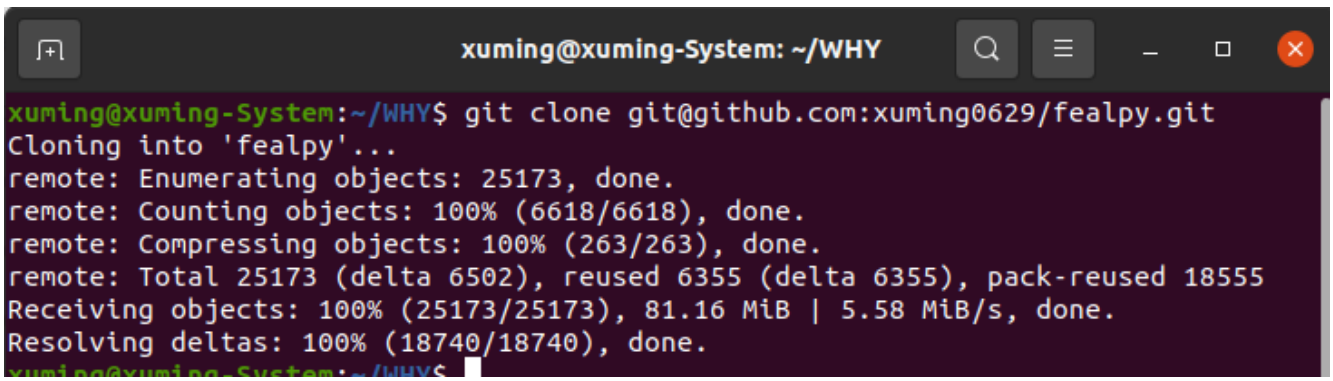
TODO：增加示例图片

1. 在原项目地址页面右上角点击 Fork 按钮，进入创建 fork 页面；
2. 在 fork 页面中可以修改仓库名、描述等等，一般可以直接保持默认；
3. 点击下方“Create fork”按钮完成创建。

（3）克隆仓库到本地

1. 将仓库通过 ssh clone 到本地

TODO: 增加图片



2. 将魏老师的 FEALPy 仓库添加为本地的 upstream 仓库

Bash

```bash
git remote add upstream git@github.com:weihuayi/fealpy.git
git remote -v
```

3. 在进行本地开发前，先将魏老师的 FEALPy 仓库与本地的 FEALPy 仓库进行融合

Bash

```bash
git fetch upstream
git merge upstream/master
```

三、操作流程

（0）明确开发内容

（1）拉取最新改动

1. 回到主分支上

Bash

```bash
git checkout master # 假设你的主分支叫做 master
```

2. 拉取

```bash
git pull
# 或者
git fetch

git merge
```

这一操作可能因网络问题失败，可以多尝试几次。

a. 若提示 up to date，说明本地的仓库已经是最新版本；
b. 若显示更新的内容，则说明更新顺利完成。

（2）本地开发

1. 创建新的分支 `feature` 用于开发（此名称仅示例，可以用其它更具代表性的名称）

```bash
git checkout -b feature
```

这条命令将创建并切换到新的分支，名为“feature”。

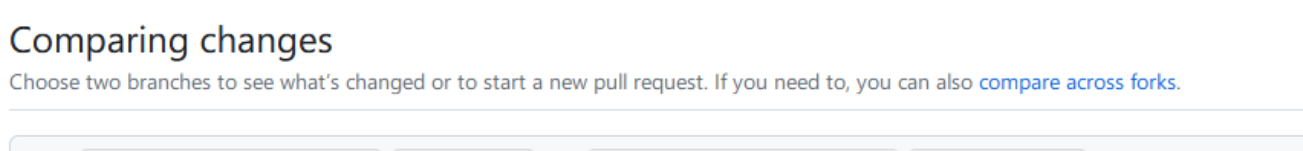
2. 软件开发
3. 上传修改到自己的仓库（注意 commit 写清楚修改的地方）

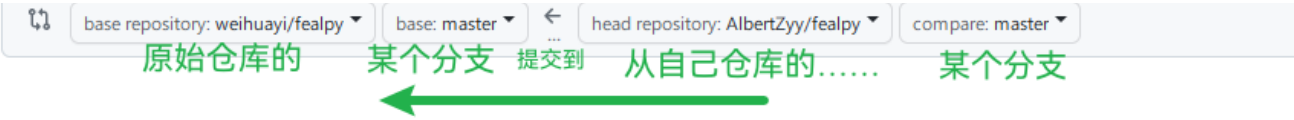
```bash
git add . # 将改动存入暂存区
git commit -m "<关于修改的关键信息>" # 提交修改
 git push # 推送到远程仓库
```

（3）发起 PR

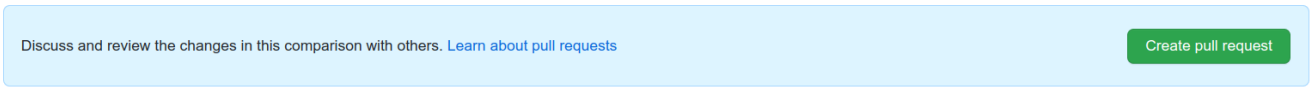
在自己 fork 的 FEALPy 仓库中，提交 pull request 给魏老师的仓库git

1. 在自己的仓库里，点击“Pull Request”选项页；
2. 点击右边的“New pull request”按钮，开始新 PR 的发起流程；
3. 选择好从自己仓库的哪个分支提交到原始仓库，然后系统会自动给出差异对比；

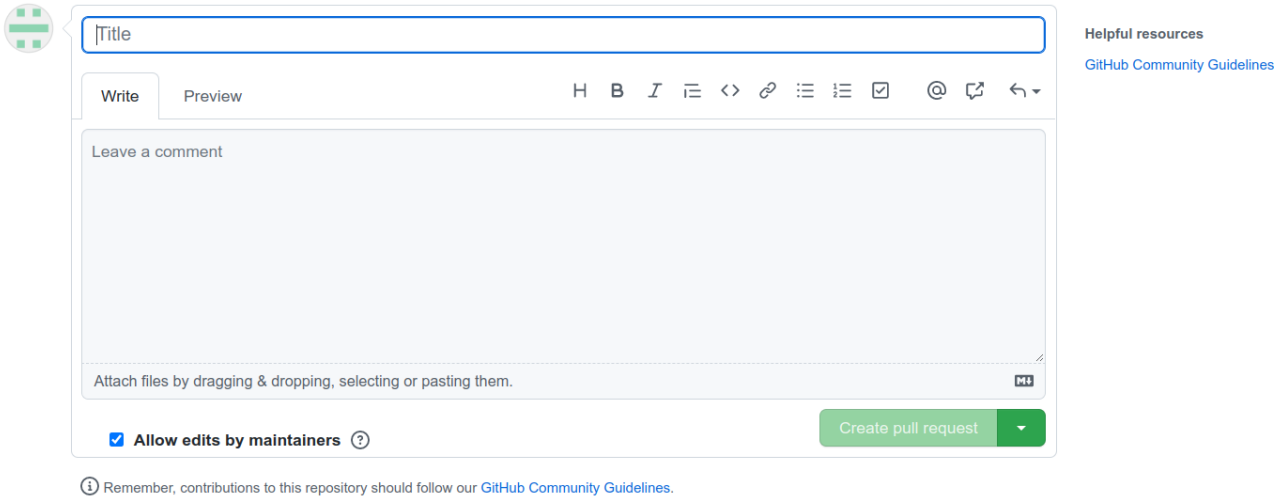




应当选择你开发时所改动的分支，如前文中的“feature”分支。



4. 在下方的 PR 标题中精简阐述修改内容，在 PR 的 comment 中列出修改项目、简要说明理由



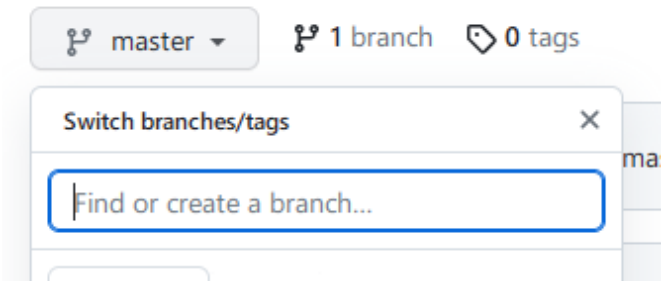
5. 点击“Create pull request”按钮发起此 PR，你可以在老师的 FEALPy 仓库的 “Pull request” 选项页看见你此次发起的 PR；

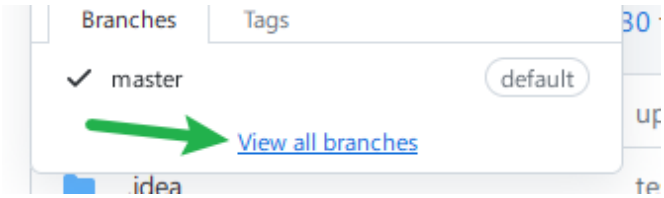
> 点击你的 PR 条目可以看见具体内容，请关注此页面你与其它开发者的互动。

6. 通过飞书提醒魏老师审核通过，将 PR 合并到 https://github.com/weihuayi/fealpy 当中。

（4）成功合并后续操作

1. 在你的 Github 仓库中删除此次开发使用的分支（它的使命已经完成），有两种方法：
   a. 在自己的仓库里查看分支，然后点击右边的删除按钮删掉；





b. 在老师的 FEALPy 仓库（即原始仓库）的“Pull request”选项页里，点击 Closed 查看所有关闭了的 PR：

找到你此次被成功合并的 PR，按下方提示按钮直接删除该分支。

> 成功合并或拒绝合并都是 Closed 状态，但成功合并的也具有 Merged 状态

2. 将原始仓库新的变化拉取到本地（即使用 git pull，注意先回到本地主分支上再 git pull）；
3. 在本地删除上次开发分支。

Bash

```bash
# 在主分支上
git branch -d feature # 假设要删除的分支名为 feature
```

🥖 以上，一次协同开发就完成了。一次次的开发、更新，就是（0）→（4）的循环。

四、注意事项

（1）

🥖 **提交 PR 后发现又需要修改?**

此时直接在这个分支上修改即可，不必重新发起 PR。PR 针对的是整个分支，而不是具体的某几次修改。所以我们在分支上提交新的修改后，PR 自然就包括这个新的提交，新的修改在合并时会被一同并入。

你与其它开发者在 PR 页面上的交流达成任何共识时，都可以在此分支里提交新的代码，更新这个 PR 所包含的代码修改。

因此，我们无法从一个分支上同时发起一个以上的 PR。如果需要同时发起两个 PR，请使用不同的分支进行开发（例如同时开发 FEALPy 的不同模块）。

---

(2)

GitHub 中 Pull Request 功能介绍

I. Pull Request 的定义

A. Pull Request 的概念

B. Pull Request 的作用

II. 创建 Pull Request

A. Fork 项目

B. Clone 项目

C. 创建分支

D. 修改代码

E. 提交 Pull Request

III. Pull Request 的审核

A. 代码审查

B. 自动化测试

C. 讨论和反馈

D. 合并代码

IV. Pull Request 的优点

A. 代码质量提升

B. 团队协作效率提高

C. 代码可追溯性增强

V. 总结

A. Pull Request 是 GitHub 中的重要功能，能够帮助团队更好地协作和管理代码。

B. 通过 Pull Request，团队可以更好地进行代码审查和测试，从而提高代码质量。

C. 同时，Pull Request 也增加了代码的可追溯性，方便团队进行后续的维护和开发。